

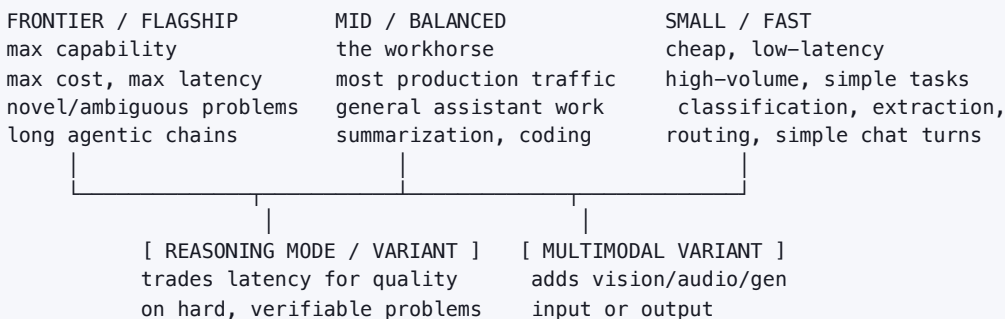
Core Concepts — The Model Landscape

Part 11 —  The LLM Model Landscape · 11-llm-models/notes/01-the-model-landscape.md

Module 04 covered how a model is built: pretraining, SFT, preference optimization, deployment. None of that is repeated here. This file assumes you know what a base model, an instruct model, and a reasoning model are, and picks up exactly where Module 04 left off: **there are now dozens of models that clear that bar, and someone has to pick one.** That someone is you.

1. The tier structure every provider converges on

Anthropic, OpenAI, Google, Mistral, and every serious open-weights lab independently arrived at the same three-tier product line, plus a cross-cutting reasoning mode. That convergence is not a coincidence — it falls out of a single economic fact: **inference cost scales with capability, and most traffic doesn't need the most capable model.**



- **Frontier/flagship** — the model a lab points to when it wants to claim state of the art. Highest cost per token, often the highest latency, and the tier you should almost never route 100% of production traffic to. Use it for the hard tail: novel reasoning, high-stakes generation, or as a "judge" model in Module 15's eval pipelines.
- **Mid/balanced** — the tier that actually pays most companies' bills. Good instruction-following, solid tool-calling, acceptable latency, and priced to run at volume. This is where you should default, and escalate up only when your eval set shows the mid tier failing a specific task.
- **Small/fast** — cheap enough and fast enough to run on every request: routing decisions, moderation, extraction, simple classification, draft-then-verify pipelines. The single highest-leverage cost lever in most LLM products, per Module 04's cost calculator, is routing the easy 80% of traffic here.
- **Reasoning mode/variant** — not really a fourth tier so much as a dial. See Module 04 §6 [04-llm/notes/01-how-llms-are-built.md] — you are buying inference-time compute, and "reasoning effort" is now a product parameter you set per request, not a model you pick once.

The mistake this section exists to prevent: defaulting to the frontier model for everything because it tested best on your one manual prompt. Frontier tax compounds — at 10x the per-token cost of a mid-tier model

across millions of requests, "it felt a bit better in the demo" is an expensive feeling.

2. Proprietary vs. open-weights — the real trade-offs

"Open" in this space is a spectrum, not a boolean, and the marketing blurs it constantly.

Axis	Proprietary API	Open-weights
Data residency / control	Data crosses to the provider's infra unless you use a private/VPC offering	Runs entirely inside your perimeter — the actual unlock for regulated data
Customization depth	Prompting, API-level fine-tuning (usually SFT-only, provider-hosted)	Full fine-tuning, continued pretraining, custom quantization, architecture surgery
Cost at scale	Per-token, no infra to run, scales linearly with usage	Fixed infra cost; cheaper past a break-even volume, worse below it (see Module 04's cost model [04-llm/notes/02-inference-and-adaptation.md] and Module 04's cost calculator [04-llm/code/llm_cost_calculator.py] for the self-hosting math)
Support / liability	A vendor SLA and someone to call	You own uptime, security patching, and incident response
Pace of improvement	Frontier labs ship first; you get day-one access	Open labs are usually 2-6 months behind frontier capability, closing over time
Licensing	A commercial ToS you accept, not negotiate (usually)	Ranges from Apache-2.0/MIT (unrestricted) to bespoke licenses with usage caps and field-of-use restrictions

The nuance that trips people up: open-weights does not mean open training data, and it does not mean unrestricted commercial use. You can download the weights of most "open" models and run them privately — that solves data residency. It does *not* tell you what data they were trained on (so you inherit unknown copyright and contamination risk), and it does not automatically grant unrestricted commercial rights: some licenses cap usage by monthly active users, forbid using the model's outputs to train a competitor, or restrict use in specific jurisdictions or industries. **Read the actual license file, every time, for every model, before you ship on it.** This is squarely Module 14 [14-ai-compliance]'s territory — treat a license review as part of model selection, not a legal afterthought bolted on after launch.

Self-hosting break-even is a real number, not a vibe: fixed GPU cost per month divided by the per-token price of the equivalent hosted tier gives you a volume threshold. Below it, self-hosting is more expensive *and* more work (patching, scaling, on-call). Module 16's AWS Bedrock track [16-ai-tech-stack/tracks/aws-bedrock] covers the middle path many teams land on: a managed host for open-weights models, trading some of the self-hosting operational burden for a per-token bill closer to (but usually still below) proprietary API pricing.

3. Benchmark literacy — read the number, then distrust it

You will be handed a leaderboard screenshot in every model conversation you ever have. Know what each number actually measures before you let it decide anything.

Benchmark	What it actually measures	Known weakness
MMLU	Multiple-choice knowledge recall across 57 academic subjects	Saturated at the frontier; measures recall, not reasoning; widely contaminated into pretraining sets
GPQA	Graduate-level science Q&A designed to resist naive search	Small (~450 questions) — a handful of wrong answers swings the score meaningfully; domain-narrow (bio/physics/chem)
SWE-bench	Whether a model's patch makes a real GitHub issue's test suite pass	Python/GitHub-specific; scores are sensitive to the agent scaffold wrapped around the model, not the model alone; leaderboard entries often aren't comparable because the scaffolds differ
LMSYS/Arena-style Elo	Human pairwise preference between two anonymized responses	Measures <i>preference</i> , not correctness — has a documented verbosity bias (humans favor longer answers even when a shorter one is more accurate), and is gameable: labs have been caught A/B-testing system prompts against live Arena traffic

The mechanism to internalize is **Goodhart's law**: *when a measure becomes a target, it ceases to be a good measure*. Three concrete ways this bites you:

1. **Contamination.** If a benchmark's questions (or close paraphrases) leaked into a model's pretraining or RLHF data, the score measures memorization, not capability. This is the exact decontamination problem from Module 04 [04-llm/notes/01-how-llms-are-built.md] at industry scale — labs decontaminate against *known* public benchmarks, which does nothing for benchmarks released after training, or for your own held-out tasks.
2. **Overfitting to popular benchmarks.** A lab optimizing post-training against public benchmark distributions produces a model that is specifically good at *that style of question*, which may or may not resemble your production traffic.
3. **The Leaderboard Illusion.** Recent analysis of Arena-style leaderboards found some vendors get preferential private testing (more variants tested before the best is submitted publicly) and disproportionate data access from the platform itself — the ranking reflects submission strategy, not just model quality. See papers/PAPERS.md [11-llm-models/papers/PAPERS.md].

The rule: a public leaderboard tells you who to shortlist. It should never tell you who to ship. Evaluate your shortlist against your own eval set, built from your own task distribution — that's Module 15 [15-ai-evals], and it's covered from the selection-framework side in notes/02 [11-llm-models/notes/02-selecting-and-migrating-models.md]. `code/benchmark_literacy_demo.py` in this module makes the contamination and sampling-variance failure modes concrete with synthetic data you can inspect.

4. Context window as a selection axis

Advertised context length is a marketing number. **Usable context is a task-dependent, empirically measured number**, and the gap between them is one of the most common sources of production disappointment.

Recall from Module 04 §7 [04-llm/notes/01-how-llms-are-built.md]: attention cost is quadratic, KV cache cost is linear, "lost in the middle" degrades retrieval accuracy for information buried mid-context, and **context rot** means quality degrades as irrelevant material accumulates even before the window is full. None of that is repeated here — but the selection implication is new:

- **Match context to the task, not to the biggest number on the spec sheet.** A support-ticket lookup that needs the last 5 messages does not benefit from a 1M-token window; it pays the latency and cost of a bigger KV cache for nothing.
- **A model advertising a huge window and a model that uses that window well are different claims.** "Needle in a haystack" retrieval tests are necessary but not sufficient — they test finding one fact, not reasoning over many facts spread through the context. Before trusting a vendor's context-length claim for your task, test it on *your* documents with *your* required reasoning, not a synthetic needle.
- **Long context does not retire RAG.** It changes the trade-off, per Module 04's conclusion: retrieval stays cheaper, fresher, more auditable, and more precise even when the window could technically fit everything.
- **Cost is not free at any window size.** Prefix caching helps only when the stable prefix is byte-identical across calls (Module 04's cost model). A huge context that changes on every call is a huge, uncached bill every time.

5. Multimodal capability as a selection axis, not a deep dive

This module treats multimodal capability the way it treats context length: as one more axis on which candidates differ, to be matched against the task. The mechanisms of diffusion models, native image/audio generation, and multimodal architectures are Module 05 [05-genai]'s territory — go there for the "how." Here, just the selection questions:

- **Vision input** (read an image/PDF/screenshot) is now table stakes across frontier and most mid-tier models. Verify OCR-quality text extraction and layout understanding on *your* document types before assuming it works — dense tables and handwriting remain weak points across the board.
- **Audio input/output** (native speech, not a bolted-on ASR/TTS pipeline) varies far more by provider and tier than vision does. If your product is voice-first, this is a first-order selection criterion, not an afterthought.
- **Native multimodal generation** (a model that generates images or audio in the same forward pass as text, rather than calling out to a separate diffusion model) is an emerging, fast-moving capability. Treat any specific claim about it as provisional and re-verify at selection time — this is exactly the kind of fact this module warns you not to trust from memory.

The selection question is never "does it support multimodal" — it's "does it support *my* modality, at the quality my task needs, at a latency and cost I can afford." Test on your actual inputs.

6. Specialization — when a smaller, narrower model wins

General-purpose frontier models are generalists by design, and generalists pay a tax: broad training data dilutes depth in any one domain. A model fine-tuned or trained specifically for code, legal text, medical text, or a narrow structured-output task can beat a much larger general model on that task, at a fraction of the cost and latency.

Signal	General-purpose model	Specialized model
Task is broad, ambiguous, or novel	Better — generalization is the point	Fragile outside its training distribution
Task is narrow and well-defined (e.g., code completion, radiology report drafting, contract clause extraction)	Adequate, but overpaying for unused breadth	Frequently wins on accuracy <i>and</i> cost
You need one model for many unrelated tasks	Simpler ops, one integration	Multiplies integrations and maintenance burden
Domain has specialized vocabulary/conventions the base model handles poorly	Prompting/RAG can partially compensate	Often the actual fix — see Module 04's fine-tuning decision table

The decision here is the same one from Module 04's adaptation-decision table [04-llm/notes/02-inference-and-adaptation.md]: try prompting a general model first, and reach for a specialized model (via fine-tuning an open-weights base, or a vendor's domain-specific offering) only once your eval set shows the general model failing in a way prompting and RAG can't fix. Specialization is a *selection* decision that follows an *adaptation* decision — don't shop for a specialized model before you've confirmed prompting the general one actually falls short.

7. Forward links

Idea here	Where it returns
Task profile -> candidate scoring	notes/02 [11-llm-models/notes/02-selecting-and-migrating-models.md], this module's code/model_selection_scorer.py
Your own eval set, not the leaderboard	Module 15 [15-ai-evals]
Self-hosting break-even, cost model	Module 04 [04-llm], Module 16 AWS Bedrock track [16-ai-tech-stack/tracks/aws-bedrock]
License review before shipping open-weights	Module 14 [14-ai-compliance]
Multimodal deep dive	Module 05 [05-genai]
Model gateway / abstraction layer	Module 10 [10-ai-architecture], continued in notes/02 [11-llm-models/notes/02-selecting-and-migrating-models.md]
Reasoning-mode mechanics	Module 04 §6 [04-llm/notes/01-how-llms-are-built.md]